CS3081 Cloud Computing Mini Project Report

Student: Kshitij Singh **Roll:** 230101049
Institution: IIIT Senapati, Manipur

Project: Knowledge Distillation and Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

Abstract

This project repository implements a teacher/student command-parsing pipeline using an Ollama Phi-3.5 teacher and an intended Pythia-70M student, with a FastAPI service, container and Kubernetes manifests, object-storage integration, network rules, tests, and CI configuration. This report separates verified code/data checks from tasks that remain unexecuted.

1. Architecture and data

The pipeline partitions natural-language commands, obtains structured labels from a teacher model, validates and cleans labels, fine-tunes a student model, and serves predictions through an API. The package contains 3,000 command-pool records with 3,000 unique IDs and 100 validation records with 100 unique IDs. The two ID sets have zero overlap. SHA-256 values are recorded in results/dataset-audit-final.json.

2. Task 1 – Teacher labeling and storage

The labeling worker supports sharding, Ollama inference, and optional S3-compatible transfer. Earlier project-operator evidence records a successful SeaweedFS 3.80 upload/download smoke test with matching file checksum. However, the complete 3,000-command teacher-labeling run and final cleaned training dataset are not present in this package snapshot.

3. Task 2 – Student training and evaluation

The training configuration targets EleutherAI/Pythia-70M with seed 42, learning rate 0.00005, batch size 1, gradient accumulation 4, two epochs, and a maximum of 1,500 examples. No data/train.jsonl, trained model, tokenizer artifact, loss history, or held-out evaluation output is available in this execution. Therefore no model accuracy or JSON-validity score is claimed.

4. Task 3 – API and containerization

The repository includes a FastAPI service, schema validation, model runner, and Dockerfiles. The service and test source are present, but a Docker image build and live endpoint checks were not performed in this execution because Docker is unavailable in the current environment.

5. Task 4 – Kubernetes

Manifests are included for namespaces, object storage, worker jobs, API Deployment/Service, Ingress, and HPA. No running k3d/Kubernetes cluster, rollout, rollback, recovery, or HPA measurement is available because kubectl and k3d are unavailable in this environment.

6. Task 5 – Network and NACL

The package contains subnetting notes, NACL design, a JSON rule file, and six sample packets. The included simulator returned 6/6 expected decisions (100%) in the final local check. NACLs are stateless, so return traffic requires explicit reverse-direction rules.

7. Task 6 – Tests and CI

The final local unit-test command returned 5 passed and 0 failed. The CI workflow is included, but a remote GitHub Actions run was not verified. The actual project repository URL, Release assets, and final tag are not available in this snapshot.

8. Task 7 – Performance, cost, and limitations

No Locust run, throughput, p50/p95 latency, CPU/memory profile, or measured deployment cost is included. The repository contains a Locust workload and cost-estimation utility, but their outputs must be generated on the target environment. The main limitations are incomplete labeling/training, unavailable container/Kubernetes tooling in this execution, and absent load-test evidence.

9. Improvements and reproducibility

Complete both teacher shards, clean and audit the full dataset, train and evaluate the student on the held-out set, build and deploy the API, test Kubernetes reliability, run Locust, and publish versioned model/tokenizer/checksum assets through a GitHub Release. Do not commit model weights. Store actual logs under results/ and tag the complete submission commit final. See README.md and docs/RUNBOOK.md for commands.

Evidence files: results/pytest-final.log, results/nacl-final.log, results/dataset-audit-final.json, and results/submission-evidence.json.